Quantstamp

Tapir Protocol

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DeFi
Timeline	2025-12-04 through 2025-12-15
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• https://github.com/tapir-protocol/tapir-core-v1 #4ec27b5 • tapir-protocol/tmarket-univ3-based #ff4d820
Auditors	• Paul Clemson Auditing Engineer • Leonardo Passos Senior Research Engineer • Andrei Stefan Auditing Engineer

Documentation quality	Undetermined
Test quality	Undetermined
Total Findings	9 Fixed: 9
High severity findings	3 Fixed: 3
Medium severity findings	3 Fixed: 3
Low severity findings	2 Fixed: 2
Undetermined severity findings	0
Informational findings	1 Fixed: 1

Summary of Findings

Tapir is a DeFi protocol that provides depeg protection for assets. It does this by allowing users to split a base asset into two equal ERC20 tokens: DP (Depeg Protection) and YB (Yield Bearer). DP tokens give users protection from potential price depegs while YB tokens offer users with a higher risk tolerance the potential to buy exposure to the Depeg Pool's underlying asset at a potential premium, and redeem them 1:1 for assets if the asset token does not experience a depeg over the pool's lifecycle. Both tokens are put into a forked Uniswap V3 pool, which has certain additional features such as dynamic fees and the ability to pause the Uniswap pool if the Depeg Pool is in a paused state.

Over the course of the audit a number of issues were found, these include logical errors within the codebase such as incorrect calculation of the protocol's earned fees (#TAP-1) as well as issues that arise due to edge cases scenarios with the project's business logic. It is recommended that the client addresses all of these issues before the protocol's launch.

Fix-Review Update 2026-01-06:
During the fix review period the Tapir team sufficiently addressed all findings highlighted in the initial report. They also addressed or acknowledged all of the auditor suggestions put forward during the audit.

ID	DESCRIPTION	SEVERITY	STATUS
TAP-1	**Incorrect Protocol Fee Calculation in the `sweepFeesToTreasury()` Function Allows `DepegPool` to Become Undercollateralized**	High	Fixed
TAP-2	**The `DepegPool` Contract's `MIN_PRICE` Constraint Can Cause a Deadlock Where the `updatePriceData()` Function Cannot Complete in the Event of a Large Depeg Event, Locking User Funds**	High	Fixed
TAP-3	**Manual Resolution of PTRW Not Possible Due To State Reversion in the `resolvePtrw()` Function, Locking User Funds**	High	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
TAP-4	`_cfg.maxPriceStaleness` May Incorrectly Include or Exclude Price Sources that Have Different Update Heartbeats	Medium	Fixed
TAP-5	`minPriceAge` Manipulation Prevents Pool Resolution	Medium	Fixed
TAP-6	Use of Yield Bearing Assets Could Cause Users Unexpected Shortcomings	Medium	Fixed
TAP-7	State Machine Transition Function Is Not Monotonic	Low	Fixed
TAP-8	Dynamic Fee Oracle Inconsistency	Low	Fixed
TAP-9	Unnecessary Storage Packing of Constant and Immutable Variables	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

In the `tapir-core` repository, all solidity files excluding those in the `interfaces/` and `mock` directories were considered within the scope of the audit.

In the `tmarket-univ3-based` repository, only the files that were altered from the original UniswapV3 codebase were included in scope.

Files Included

Repo: `https://github.com/tapir-protocol/tapir-core-v1`

Included Paths: `contracts/`

Extensions: `sol`

Repo: `https://github.com/tapir-protocol/tmarket-univ3-based`

Included Paths: `contracts/UniswapV3Factory.sol`, `contracts/UniswapV3Pool.sol`, `contracts/UniswapV3PoolDeployer.sol`

Extensions: `sol`

Operational Considerations

The following assumptions are made of the codebase:

- The `DepegPool` should not use rebasing tokens as the underlying `ASSET` as the pool is unable to properly handle additional tokens in rebasing scenarios.
- In `DepegPool` it is assumed that all addresses added to the `isAuthorizedRouter` mapping are instances of `TapirRouter`, as a malicious address present here may be able to burn a user's `YB_ASSET` and `DP_ASSET` against their will.
- In `DepegPool` the success fee is applied when the price of the underlying asset has increased is based on the start price of the asset when the pool was created, not the price when the user originally deposited their assets. This could cause users to pay a "success fee" even if the price of the underlying asset has not changed since they called the `splitTokens()` function.
- Addresses with the operator role in `TapirOracle` are trusted to ensure that the most recent prices are available before providing data to the `DepegPool`.
- The protocol inherits the security considerations of the each of the external protocols used to provide price fees.
- The protocol inherits the security considerations of Pendle when using an Oracle that interacts with the Pendle protocol.
- The audit assessed only the changes made to the underlying Uniswap V3 codebase and assumed that the unchanged features of the forked codebase were safe.
- Users must be aware of the potential risks of trapped funds when interacting with `DepegPool` instances where the underlying `ASSET` is not a censorship resistant token, such as those with blacklists or the ability to freeze address balances.
- Both the UniswapV3 fork and DepegPool implement instantly modifiable fee parameters without timelock protection. The `tapirAdmin` can change `dynamicFee` from 0.05% to 1% instantly, enabling MEV extraction within user slippage tolerance. Similarly, `OPERATOR_ROLE` can change `redemptionFeeBp` and `successFeeBp` during REDEMPTIONS state without user protection mechanisms, as redemptions lack `minOut` parameters. This creates front-running opportunities for compromised admins or operators to extract unexpected fees from user transactions.
- The `tapirAdmin` in UniswapV3 pools is immutable, preventing key rotation, compromise recovery, or governance transitions to DAOs. Once deployed, if the admin key is lost or compromised, no recovery mechanism exists. Similarly, there's no path to decentralization or multisig upgrades without deploying new pools. This creates permanent operational risk and requires secure key management from deployment onward.
- The UniswapV3 fork pause only blocks swaps (not liquidity operations) while DepegPool pause blocks all user operations (split, unsplit, redeem). However, `resolvePriceDepeg()` lacks `whenNotPaused` modifier, allowing state progression while pool is paused. This creates scenarios where pool advances to REDEMPTIONS state but users cannot redeem, breaking admin emergency response capabilities.
- The `resolvePriceDepeg()` function requires `finalPriceData` to be at least `minPriceAge` old before resolution (default 4 hours to 30 days range). This prevents manipulation through fresh oracle updates but creates timing dependencies between oracle submissions and resolution execution. Combined with mutable `minPriceAge` parameter, creates risk of indefinite pool lockup if admin increases age requirement after oracle submission.
- UniswapV3 pools maintain two separate fee values: immutable `fee` (for pool identification, CREATE2 salt, tick spacing) and mutable `dynamicFee` (for actual swap costs). While `dynamicFee` initializes to `fee` value, they can diverge completely. Oracle integrators and external protocols must understand to use `fee()` for pool identification but `dynamicFee()` for swap cost calculations. TWAP price oracles are unaffected as they track prices, not fees.
- Fees accumulate in DepegPool from unsplit and redemption operations, remaining in the contract until explicitly swept via `sweepFeesToTreasury()`. The function is permissionless and can be called anytime, calculating fees based on the difference between contract balance and token obligations. Proper accounting must maintain invariant: `poolBalance >= dpSupply + ybSupply + accumulatedFees`.
- It is assumed that all centralized actors are trusted, their private keys are secured using industry best practices (e.g., use of multisig-based wallets).
- The oracle operator is expected to invoke the `checkpoint()` in a timely fashion to ensure data is written into the ring buffer, therefore setting the HWM, closing price, and current price. The operator shall not delay, skip, or strategically time checkpoints to bias HWM or resolution prices.
- In xChain mode, all oracle outputs depend on external bridge messaging. Delayed, censored, or reordered messages may cause stale or incorrect price data to be written to the DepegPool. Protocol liveness is therefore partially dependent on external infrastructure. The latter, however, is assumed to operate fully, without errors, and without significant latencies.

Key Actors And Their Capabilities

The protocol includes the following actors:

1. **Users**: Can interact with DepegPool during ACTIVE state to split base assets into DP and YB tokens via `splitToken()` receiving equal amounts of both tokens (1 ASSET → 0.5 DP + 0.5 YB). Users can unsplit DP and YB tokens back to base assets via `unSplitTokens()` during ACTIVE state, subject to success fees on profit and redemption fees on base amount. During REDEMPTIONS state, users can redeem DP and YB tokens via `redeemTokens()` based on depeg resolution outcome (if depegged, DP gains value and YB loses value proportionally). Users can interact through authorized routers which act on their behalf if they've approved tokens to the DepegPool. Users can trade DP/YB tokens on the modified UniswapV3 pools when not paused.
2. **tapirAdmin (UniV3 Fork)**: Immutable privileged address for each UniswapV3 pool deployed via factory with `tapirAdmin` parameter. Can instantly change `dynamicFee` from 0 to 10,000 pips (0-1%) with no timelock via `setFee()` affecting all swap operations. Can instantly pause and unpause the pool via `setPaused()` blocking all swap operations while allowing liquidity management (mint, burn, collect) to continue. Has complete control over pool trading operations but cannot modify liquidity positions or transfer ownership as role is immutable.
3. **DEFAULT_ADMIN_ROLE (DepegPool)**: Highest authority role in DepegPool granted to pool owner at deployment. Can authorize and revoke router addresses via `setAuthorisedRouter()` granting global permission to act on behalf of all users. Can change mutable time parameters including `cooldownDuration` (4 hours to 30 days) and `minPriceAge` (4 hours to 30 days) at any time without state guards. Can initiate oracle changes with 7-day timelock via `proposeOracleChange()` and `executeOracleChange()`. Can pause and unpause all pool operations. Can set redemption and success fee rates. Has authority to grant OPERATOR_ROLE to additional addresses.
4. **OPERATOR_ROLE (DepegPool)**: Operational hot wallet role with subset of admin capabilities. Can set redemption fee rates and success fee rates instantly without state guards via `setRedemptionFeeBp()` and `setSuccessFeeBp()`. Can pause pool operations (but only admin can unpause). Can rescue ERC20 tokens from the contract with restrictions on core tokens (DP, YB, base asset) requiring 30-day timelock. Multiple operators can exist simultaneously for operational flexibility.
5. **Authorized Routers**: Addresses approved via `isAuthorisedRouter` mapping that can execute split, unsplit, and redeem operations on behalf of ANY user who has approved tokens to the DepegPool. Authorization is global (not per-user), meaning one compromised router affects all users. TapirRouter is typically the primary authorized router handling multi-step operations including split+buy and sell+unsplit sequences across DepegPool and UniswapV3 pools.
6. **Oracle**: Single address (or cross-domain messenger on L2) with exclusive permission to update prices. Can update `currentPrice` anytime via `updateCurrentPrice()` affecting success fee calculations. Can update `finalPriceData` (hwmPrice and resolutionPrice) during COOLDOWN state via `updatePriceData()` which determines depeg resolution outcome. Oracle updates are critical for pool progression and depeg value calculations.

Findings

TAP-1

Incorrect Protocol Fee Calculation in the `sweepFeesToTreasury()` Function Allows `DepegPool` to Become Undercollateralized

High Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `9cbbf29523453e709a227c1a16fa8340c86fbcd2` .
Additional fix in: `2c0cc1e7bfef89bc39e28e5a0042cf209a665264`

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: The logic of the `sweepFeesToTreasury()` function calculates how many of the `ASSET` tokens held in the contract can be transferred to the treasury as earned fees. However, due to an error in how this amount is calculated, `feeAmount` will be half of the total `ASSET` tokens in the contract. This is because `dpSupply` and `ybSupply` are expected to be 50% of the contract's total `ASSET` balance. As this function is permissionless and can be called at state in the protocols lifecycle, there is a significant risk of griefing where funds can be repeatedly forced to the treasury, leaving the protocol under-collateralized.

Exploit Scenario:
Consider the following example:

1. `ASSET` is USDC
2. 100,000 USDC tokens are deposited into the protocol via the `splitTokens()` function
3. Therefore `dpSupply == 50,000` and `ybSupply == 50,000`
4. `sweepFeesToTreasury()` will calculate `baseInContract` (100,000) minus `maxSupply` (50,000) and transfer the remaining 50,000 USDC to the treasury, leaving the protocol insolvent
5. If the treasury attempts to return the funds to the contract, the attacker can repeatedly call the permissionless `sweepFeesToTreasury()` function

Recommendation: Consider calculating the total value (in `ASSET`) of the total supply of both the DP and YB tokens, and only allowing redemption of the excess. Additionally, consider only allowing this function to be called during `STATE_REDMPTIONS` .

TAP-2

The `DepegPool` Contract's `MIN_PRICE` Constraint Can Cause a Deadlock Where the `updatePriceData()` Function Cannot Complete in the Event of a Large Depeg Event, Locking User Funds

High Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4155f0a04ec0913274fe906687e7f4fadd541827`.

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: The `DepegPool` contract has a `MIN_PRICE` that the underlying asset must be when the `TapirOracle` provides the final price data via the `updatePriceData()` function. However, if there is a significant depeg event there is no guarantee that the price of the underlying asset will be greater than `MIN_PRICE`. In this scenario, the `DepegPool` contract will not be able to progress into the `REDEMPTION` stage, preventing users from withdrawing their funds.

Recommendation: Consider allowing the `TapirOracle` to provide any `_resolutionPrice`.

Additionally, ensure the formulas used in `DepegPool` to calculate how funds are dispersed between `DP_ASSET` tokens and `YB_ASSET` tokens during the `REDEMPTION` stage corre handle severe depeg events, to ensure redemptions are handled fairly.

TAP-3
Manual Resolution of PTRW Not Possible Due To State Reversion in the `resolvePtrw()` Function, Locking User Funds

High | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `c97e619c2626942fb5fe3658c1e1abf5621bcaed`.

File(s) affected: `tapir-core-v1/contracts/TapirPtrwOracle.sol`

Description: In the `resolvePtrw()` function, if the attempt to redeem PT tokens from Pendle fails (no tokens are redeemed), a manual price resolve is required. However, in the current implementation, the `resolvePtrw()` function immediately reverts after setting `forceManualResolve = true`, rewinding all state transitions made during the function call, including the setting of the `forceManualResolve` boolean.

This means that in scenarios where a manual resolution is necessary, if the `pendlePT` balance of the `TapirPtrwOracle` contract is non-zero it will not be possible to execute a manual resolution, leaving the oracle contract in a deadlocked state where the oracle contract cannot write price data to the `DepegPool` contract.

Recommendation: When a manual resolution is required, consider the following changes:

```
if (ptrwArv == 0) {
    forceManualResolve = true;
    emit ManualResolveRequired();
    return; // Return before ptrwResolved is set to true outside this conditional block
}
```

TAP-4
`_cfg.maxPriceStaleness` May Incorrectly Include or Exclude Price Sources that Have Different Update Heartbeats

Medium | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `a4f5bed63ebe48386a87a253b67776fbee0c9f1e`.

File(s) affected: `tapir-core-v1/contracts/TapirOracle.sol`

Description: In `TapirOracle.checkpoint()`, checks are made to ensure the price provided by an oracle has been updated within a specific `maxPriceStaleness`. However, different price feeds will have different update tolerances (heartbeats), where if the price has not moved outside of the accepted tolerance, the oracle will not be updated unnecessarily until the heartbeat time. This could lead to the `checkpoint()` function mistakenly considering a price stale while it is still actually valid. Alternatively, if the `maxPriceStaleness` was too large for a specific price source, a stale price might be considered valid by the Tapir oracle.

Recommendation: Consider storing individual `maxPriceStaleness` values within the `priceSources` struct that accurately reflects the update heartbeat of the underlying oracle.

TAP-5 `minPriceAge` Manipulation Prevents Pool Resolution

Medium | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `e16dda7075ac06c0d8bb60820e47ccbae14020a6` .

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: The `minPriceAge` parameter is mutable via `setMinPriceAge()` without state guards. The `resolvePriceDepeg()` function requires `block.timestamp - finalPriceData.timestamp >= minPriceAge` before allowing resolution. When an admin increases `minPriceAge` (up to MAX_DURATION = 30 days) while the pool is in RESOLUTION state, the function reverts with `PriceDataTooRecent`, indefinitely blocking progression to REDEMPTIONS and effectively locking user funds. Unlike HIGH-01 (backwards transition), this freezes the state machine in RESOLUTION.

Recommendation: Add a state guard sanity check:

```
function setMinPriceAge(uint256 _minAge) external onlyRole(DEFAULT_ADMIN_ROLE) {
    uint8 state = getState();
    require(state == STATE_ACTIVE || state == STATE_COOLDOWN,
            "Can only change before RESOLUTION");
    if (_minAge < MIN_DURATION || _minAge > MAX_DURATION) revert MinAgeOutOfBounds();
    minPriceAge = _minAge;
}
```

TAP-6
Use of Yield Bearing Assets Could Cause Users Unexpected Shortcomings

Medium | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `ea4c9f9c7f6892f7670c3dffe4ac77c66110f59e` .

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: When the `ASSET` token of a `DepegPool` instance is a yield bearing asset users are likely to face a `successFee` upon redemption which takes a percentage fee of the users redeemable amount based on the price increase of the underlying asset from the `startingPrice` when the pool was deployed. For users who deposit towards the end of the pools `activeDuration` they will still be forced to pay the full `successFee` even though the majority of the value accrual occurred before the user's assets were added to the pool. This potential risk is not clearly outlined to the user who may end up with less `ASSET` tokens than they originally deposited due to the `successFee`.

Recommendation: Consider either of the following options:

1. Remove the `successFee` from the protocol as it is difficult to enforce fair fees based on the amount of time the user was present in the pool and potentially disincentivises users from participanting in the pool.
2. Provide public-facing documentation to capture which tokens shall be supported. For each token, provide an explanation of what the user is protected against, and what potential losses he may experience.

TAP-7 State Machine Transition Function Is Not Monotonic

Low | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `3adbf035b2f0d537d5417c42333bddbfe983135b` .

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: The protocol expected a strict linear progression from the state machine as follows: 1 (`STATE_ACTIVE`) → 2 (`STATE_COOLDOWN`) → 3 (`STATE_RESOLUTION`) → 4 (`STATE_REDEMPTIONS`). However, because the protocol's admin is able to change the `cooldownDuration` at any time, the project could theoretically move from a later state back to `STATE_COOLDOWN`. Due to how the current state is assessed in the `getState()` function, it is possible to end up with a transition from `STATE_RESOLUTION` → `STATE_COOLDOWN` (3 → 2) and from `STATE_REDEMPTIONS` → `STATE_COOLDOWN` (4 → 2).

Recommendation: Consider the following:

- Record the present state as an enum.

- Replace the current `getState()` function with one that gets or increments the state as necessary, for example `updateAndGetState()`. This can ensure that the state must progress forward sequentially and avoid any possible unforeseen backwards state transitions.

TAP-8 Dynamic Fee Oracle Inconsistency

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `3b8853fbeb4fb6b3a9f753343254a3b6cc92339f` .

File(s) affected: `tmarket-univ3-based/contracts/UniswapV3Pool.sol`

Description: Pools used as price oracles should document the dual-fee system:

```
uint24 public immutable override fee;        // Pool identifier
uint24 public override dynamicFee;           // Actual swap fee
```

For Oracle Integrators:

- Use `fee()` for pool identification and tick spacing
- Use `dynamicFee()` for actual swap cost calculations
- TWAP calculations are unaffected (based on price, not fees)

Recommendation: Add specifics of the changes made to the core UniswapV3 protocol to the project's integration documentation.

TAP-9
Unnecessary Storage Packing of Constant and Immutable Variables

Informational Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `fb5fe0ba5ff865e08d10e4ef4bdb61aba69b9de5` .

File(s) affected: `tapir-core-v1/contracts/DepegPool.sol`

Description: The `DepegPool` contract packs its storage variables for a more efficient storage layout. However, it also unnecessarily packs variables marked as `constant` and `immutable`. This is unnecessary as both constant and immutable variables in solidity are compiled directly into the contract's bytecode rather than being held in the contract's storage onchain.

Recommendation: Consider reorganising the DepegPools storage layout, ignoring any variables that will not be held in storage on chain.

Auditor Suggestions

S1 Flash Loan Removal May Breaks Composability

Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `f4bd1d4941173f989dda3c8482a5666d5adb9d9e` .

File(s) affected: `tmarket-univ3-based/contracts/UniswapV3Pool.sol`

Description: Flash loan (`flash()`) function removed to reduce contract size under 24kb limit (from docs):

"Tapir AMM removes flash-loan swaps to reduce contract size under the 24kb limit."

Possible Impact on Ecosystem:

1. Arbitrage bots relying on flash loans
2. Liquidation bots using flash loans
3. Protocols expecting standard UniV3 interface
4. Advanced DeFi strategies requiring atomic flash loans

Recommendation: Ensure both external and inline documentation clearly documents the changes made to the underlying UniswapV3 codebase so those interacts with it can be reasonably informed of the changes made.

S2 General Improvements

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `0a31d2912a8403447e4d32a53e9076c9e02703ea` .

File(s) affected: `tapir-core-v1/contracts/*`

Description: The following improvements would improve the overall quality of the codebase:

- The `DepegPool` has a large number of arguments, which decreases the readability of the codebase. Consider refactoring these arguments into a `DepegPoolParams` struct and/or pairing related variables (such as `_ybName` and `_ybSymbol`) into a more general struct (such as `YbMetadata`).
- The `DepegPool` has multiple internal calculation functions, most are named `_calculateX()` , however `_calcRedeemValues()` does not follow this naming convention. Consider changing this to `_calculateRedeemValues()` to be consistent with the naming conventions of the rest of the codebase.
- The `_calcRedeemValues()` function takes one argument `_principalDepeggedValue` , however when this function is used in the contract the value passed is always the state variable `principalDepeggedValue` . Consider reading this value directly within `_calcRedeemValues()` rather than taking it as an argument.
- The `DepegPool.redeemTokens()` function checks if `_amountDP > type(uint256).max / principalDepeggedValue` . Given the total supply of DP tokens cannot be greater than 50% of the total supply of the underlying asset, it is very unlikely this check will ever come into effect, and if it did the function would revert anyway. Consider removing this check to improve the readability of the codebase.
- The `DepegPool.redeemTokens()` function checks if `_amountYB > type(uint256).max / (BP_IN_INTEGER - principalDepeggedValue)` . Given the total supply of YB tokens cannot be greater than 50% of the total supply of the underlying asset, it is very unlikely this check will ever come into effect, and if it did the function would revert anyway. Consider removing this check to improve the readability of the codebase.
- In `DepegPool.executeOracleChange()` , the `pendingOracle` storage variable is read from the contract's state three times. Consider caching the variable in memory at the start of the function's logic to reduce storage read costs.
- In `TapirOracle.recordApi3Price()` , the `_sources` storage variable is read from the contract's state multiple times. Consider caching the variable in memory at the start of the function's logic to reduce storage read costs.
- In `TapirOracle.recordChainlinkPrice()` , the `_sources` storage variable is read from the contract's state multiple times. Consider caching the variable in memory at the start of the function's logic to reduce storage read costs.
- In `TapirOracle.recordRedStoneClassicPrice()` , the `_sources` storage variable is read from the contract's state multiple times. Consider caching the variable in memory at the start of the function's logic to reduce storage read costs.
- In `TapirOracle.recordTellorPrice()` , the `_sources` storage variable is read from the contract's state multiple times. Consider caching the variable in memory at the start of the function's logic to reduce storage read costs.
- In `TapirOracle.checkpoint()` two variables (`nowTs` and `asOfTs`) both cache `block.timestamp` . Consider removing on of these variables.
- In `TapirOracle._getRecentCheckpointCount()` the storage variable `_count` is read multiple times in a loop. Consider caching `_count` in memory and then looping over the memory variable to reduce storage read costs.

Recommendation: Consider implementing the suggested changes.

S3 Some Comments Do Not Match Contract Behaviour

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `07dbf50245dc5bc53e37c7cb39c068d16ea0bd26` .

File(s) affected: `tapir-core-v1/contracts/*`

Description: There are multiple places across the `DepegPool` and `TapirOracle` contracts where the documentation, comments, or high-level descriptions do not fully match what the code actually does.

Recommendation: Consider a thorough review of the codebases documentation to ensure the expectations outlined matches the actions of the codebases logic.

S4 Renouncable Ownership Should Be Avoided

Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:

> `Issue acknowledged and internal operational guidelines will reflect the risk.`

File(s) affected: `tapir-core-v1/contracts/DepegFactory.sol`

Description: The `DepegFactory` inheris the `Ownable` contract from the OpenZepplin library, this contract allows the contract's ownership to be renounced. If this were to happen a new factory instance would have to be deployed in order to create new `DepegPool` instances. While not a security concern, stopping this from occurring is preferred.

Recommendation: Consider overriding the inherited OpenZepplin `renounceOwnership()` function to revert when called.

S5 Errors are not Parameterized to Capture Underlying Context

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `0bc0a5835a2f7fcb4dffbfc29c2a942d1f29a480` .

File(s) affected: `tapir-core-v1/contracts/DepegFactory.sol`, `tapir-core-v1/contracts/DepegPool.sol`

Description: The errors in DepegFactory are not parameterized to capture the context that led to the reported errors. Aiding context to errors increase debugging information crutial for spotting the root cause of failed transactions.

Recommendation: Consider reviewing the errors used in the contract and include additional context wherever possible.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/tapir-protocol/tapir-core-v1`

- `249...cc7 ./contracts/DepegFactory.sol`
- `4fa...58c ./contracts/DepegPool.sol`
- `c14...d24 ./contracts/DepegToken.sol`
- `cab...dc1 ./contracts/TapirOracle.sol`
- `5e7...1f0 ./contracts/TapirPtrwOracle.sol`
- `bee...7ac ./contracts/TapirRouter.sol`
- `d67...03d ./contracts/libraries/TimeDecay.sol`

- `4a1...b5a ./contracts/libraries/UniswapV3Math.sol`

  Repo: `https://github.com/tapir-protocol/tmarket-univ3-based`

- `8c4...425 ./contracts/UniswapV3Factory.sol`
- `33c...92b ./contracts/UniswapV3Pool.sol`
- `666...973 ./contracts/UniswapV3PoolDeployer.sol`

Test Suite Results

To run the project's test suite use the following commands:

```
npm install
npx hardhat test
```

```
  DepegFactory
    Deployment
      ✔ Should set the correct owner
    Owner-only Access Control
      Owner can deploy pools
        ✔ Should allow owner to deploy a DepegPool
        ✔ Should emit DeployDepeg event when owner deploys
        ✔ Should allow owner to deploy multiple pools
      Non-owners cannot deploy pools
        ✔ Should revert when user1 tries to deploy a pool
        ✔ Should revert when user2 tries to deploy a pool
        ✔ Should revert when non-authorized user tries to deploy a pool
      Ownership Transfer
        ✔ Should allow new owner to deploy after ownership transfer
        ✔ Should prevent old owner from deploying after ownership transfer
        ✔ Should prevent non-owner from transferring ownership
      Edge Cases
        ✔ Should prevent deployment after ownership renouncement
        ✔ Should maintain access control across multiple users
    View Functions
      ✔ Should return correct depeg module after deployment
      ✔ Should track multiple deployed pools correctly
    Cross-Domain Messenger Configuration
      ✔ Should allow setting xDomainMessenger to zero address (native-chain mode)
      ✔ Should allow setting xDomainMessenger to non-zero address (cross-chain mode)

  DepegPool
    Deployment & Initialization
      ✔ Should deploy with correct initial parameters
      ✔ Should set correct token addresses
      ✔ Should deploy valid ERC20 tokens as DP & YB
      ✔ Should grant correct roles
      ✔ Should start in ACTIVE state
      Constructor Validation
        ✔ Should revert when success fee is too high
        ✔ Should revert when minPriceAge is too short
        ✔ Should revert when minPriceAge is too long
        ✔ Should accept minPriceAge at the minimum boundary
        ✔ Should accept minPriceAge at the maximum boundary
        ✔ Should accept successFeeBp at the maximum boundary
        ✔ Should accept zero successFeeBp
    State Management
      ✔ Should transition from ACTIVE to COOLDOWN after pool duration
      ✔ Should remain in ACTIVE state if duration not elapsed
      ✔ Should transition from COOLDOWN to RESOLUTION after cooldown duration when price data is updated
      ✔ Should transition from RESOLUTION to REDEMPTIONS after min price age & resolution price set
      ✔ Should remain in RESOLUTION state if min price age not elapsed
      ✔ Should not enter RESOLUTION state if resolution price not set
      ✔ Should remain in COOLDOWN state if final price data is not set, even after cooldown duration
passes
```

```
    ✔ Should progress to RESOLUTION when final price data is set AFTER cooldown duration has passed
  splitToken
    ✔ Should split tokens correctly in ACTIVE state
    ✔ Should emit SplitToken event
    ✔ Should revert when splitting zero amount
    ✔ Should revert when not in ACTIVE state
    ✔ Should revert when paused
    ✔ Should not revert after unpausing
    ✔ Should handle odd amounts correctly (rounding down)
    ✔ Should not allow splitting a different counterparty's tokens
    ✔ Should allow an authorised router to split a different counterparty's tokens
  unSplitTokens
    ✔ Should unsplit tokens correctly
    ✔ Should emit UnSplitTokens event
    ✔ Should revert when unsplitting zero amount
    ✔ Should revert when not in ACTIVE state
    ✔ Should charge success fee when price has increased
    ✔ Should revert when not called by counterparty or authorised router
    ✔ Should allow an authorised router to unsplit a different counterparty's tokens
  Admin & Operator Functions
    setRedemptionFeeBp
      ✔ Should allow admin to set redemption fee
      ✔ Should allow operator to set redemption fee
      ✔ Should revert when non-authorized user tries to set fee
      ✔ Should emit RedemptionFeeBpUpdated event
      ✔ Should accept a zero redemption fee
      ✔ Should reject when redemption fee is too high
    setSuccessFeeBp
      ✔ Should allow admin to set success fee
      ✔ Should allow operator to set success fee
      ✔ Should revert when non-authorized user tries to set fee
      ✔ Should emit SuccessFeeBpUpdated event
      ✔ Should allow zero success fee
      ✔ Should revert when fee exceeds maximum
    setCooldownDuration
      ✔ Should allow admin to set cooldown duration
      ✔ Should revert when cooldown is too short
      ✔ Should revert when cooldown is too long
      ✔ Should revert when operator tries to set cooldown
      ✔ Should revert when non-authorized user tries to set cooldown
      ✔ Should emit CooldownDurationUpdated event
    setMinPriceAge
      ✔ Should allow admin to set min price age
      ✔ Should revert when min age is too short
      ✔ Should revert when min age is too long
      ✔ Should emit MinPriceAgeUpdated event
      ✔ Should revert when non-authorized user tries to set min price age
      ✔ Should revert when operator tries to set min price age
    pause/unpause
      ✔ Should allow admin to pause
      ✔ Should allow operator to pause
      ✔ Should allow admin to unpause
      ✔ Should not allow operator to unpause
      ✔ Should prevent operations when paused
      ✔ Should record last pause timestamp
    rescueErc20
      ✔ Should allow rescuing non-core tokens immediately
      ✔ Should prevent rescuing core tokens too early
      ✔ Should allow rescuing core tokens 30 days after resolution
      ✔ Should revert when rescuing zero amount
      ✔ Should revert when trying to rescue a zero token
  Oracle Management
    proposeOracleChange
      ✔ Should allow admin to propose oracle change
      ✔ Should set the oracleChangeTimestamp correctly into the future
      ✔ Should revert when proposing zero address
      ✔ Should revert when proposing non-contract address
      ✔ Should revert when operator tries to propose
      ✔ Should revert when non-authorized user tries to propose
    executeOracleChange
      ✔ Should revert when no oracle change pending
      ✔ Should revert when timelock not expired
```

```
    ✔ Should not revert when timelock has expired
    ✔ Should allow anyone to execute oracle change
    ✔ Should reject trying to execute oracle change twice
  Cross-Domain Oracle Calls (L2)
    updateCurrentPrice via xDomain
      ✔ Should allow oracle to update price via cross-domain messenger
      ✔ Should revert when called directly by oracle (not via messenger)
      ✔ Should revert when called via messenger but wrong L1 sender
      ✔ Should revert when called from non-messenger contract
    updatePriceData via xDomain
      ✔ Should allow oracle to update price data via cross-domain messenger
      ✔ Should revert when called directly by oracle (not via messenger)
      ✔ Should revert when called via messenger but wrong L1 sender
      ✔ Should revert when called from non-messenger contract
    L2 vs L1 mode comparison
      ✔ Should use L1 mode (direct oracle calls) when xDomainMessenger is zero address
      ✔ Should use L2 mode (cross-domain calls) when xDomainMessenger is set
Redemption Flow
  ✔ Should revert redemption when not in REDEMPTIONS state
  ✔ Should revert when not called by counterparty or authorised router
  ✔ Should allow an authorised router to redeem tokens for a different counterparty
  ✔ Should should return 1:1 when no depeg occurred
  ✔ Should return correctly if a depeg has occurred
  ✔ Should work even with zero DP or YB tokens
resolvePriceDepeg
  ✔ Should revert when not in RESOLUTION state
  ✔ Should revert if finalPriceData is not set
  ✔ Should revert if finalPriceData is too recent
  ✔ Should set poolHasDepegged to false if no depeg occurred
  ✔ Should set poolHasDepegged to true if depeg occurred
  ✔ Should correctly calculate depeg size
  ✔ Should emit DepegPriceResolved event
  ✔ Should be callable by anyone
sweepFeesToTreasury
  ✔ Should sweep fees to treasury
  ✔ Should emit TreasuryFeeCollected event
setAuthorisedRouter
  ✔ Should allow admin to authorise a router
  ✔ Should allow admin to revoke an authorised router
  ✔ Should allow multiple routers to be authorised simultaneously
  ✔ Should not allow anyone else to set authorised router
Access Control
  ✔ Should have DEFAULT_ADMIN_ROLE set correctly
  ✔ Should have OPERATOR_ROLE set correctly
  ✔ Should allow admin to grant OPERATOR_ROLE
  ✔ Should allow admin to revoke OPERATOR_ROLE
  ✔ Should allow admin to reassign DEFAULT_ADMIN_ROLE
  ✔ Should allow admin to revoke DEFAULT_ADMIN_ROLE
Edge Cases & Security
  ✔ Should handle very small amounts
  ✔ Should handle very large amounts
Hardcoded constants
  ✔ Should have correct BP_IN_INTEGER
  ✔ Should have correct MIN_DURATION
  ✔ Should have correct MAX_DURATION
  ✔ Should have correct ORACLE_CHANGE_DELAY
  ✔ Should have correct MAX_SUCCESS_FEE_BP
  ✔ Should have correct successFeeBp (constructor hardcoded)
  ✔ Should have correct minPriceAge (constructor hardcoded)
View Functions
  ✔ Should return correct pool name and flag
  ✔ Should return correct token addresses
  ✔ Should return correct price data
  ✔ Should return correct durations
  ✔ Should return correct fee rates
6-Decimal Base Token Support
  ✔ Should create DP/YB tokens with 6 decimals matching base asset
  ✔ Should split 6-decimal tokens correctly
  ✔ Should unsplit 6-decimal tokens correctly
  ✔ Should handle complete lifecycle with 6-decimal tokens and no depeg
  ✔ Should handle 6-decimal tokens with depeg scenario
  ✔ Should calculate fees correctly with 6-decimal precision
```

```
    Complete Lifecycle Tests
      No Depeg Scenario
        ✔ Should handle complete lifecycle when no depeg occurs
      With Depeg Scenario
        ✔ Should handle complete lifecycle with 10% depeg
        ✔ Should handle minimal depeg below 0.1% threshold
        ✔ Should calculate depeg size correctly for various depeg percentages
      Multiple Users Interaction
        ✔ Should handle multiple users splitting and redeeming
    Invariant Tests
      ✔ Should maintain supply invariant: yb.totalSupply() == dp.totalSupply() ==
baseAsset.balanceOf(depegPool) / 2
      ✔ Should maintain solvency invariant: yb.totalSupply() * ybValue + dp.totalSupply() * dpValue <=
baseAsset.balanceOf(depegPool)

  DepegToken
    Deployment
      ✔ Should have correct name & symbol
      ✔ Should have correct decimals
      ✔ Should have correct decimals for 6 decimal token
      ✔ Should set DepegPool as the deployer
      ✔ Should have zero initial total supply
    Minting
      ✔ Should be mintable by the DepegPool
      ✔ Should not be mintable by anyone other than the DepegPool
      ✔ Should correctly update total supply after minting
      ✔ Should emit Transfer event when minting
    Burning
      ✔ Should be burnable by the DepegPool
      ✔ Should not be burnable by anyone other than the DepegPool
      ✔ Should correctly update total supply after burning
      ✔ Should emit Transfer event when burning
    Access Control
      ✔ Should correctly identify DepegPool

  Multi-Decimal Integration Test
    Decimal Configuration Validation
      ✔ Should have correct asset decimals
      ✔ Should have DP/YB tokens with same decimals as base asset
      ✔ Should have correct oracle source decimals configured
      ✔ Should have pool prices configured in base asset decimals
    Oracle Decimal Normalization
      ✔ Should handle API3 price with matching decimals (6 to 6, no normalization)
      ✔ Should normalize Chainlink price from 8 to 6 decimals
      ✔ Should normalize RedStone price from 18 to 6 decimals
      ✔ Should handle different prices correctly with decimal normalization
      ✔ Should create checkpoint with median of normalized prices
      ✔ Should handle different decimal configurations correctly
    Complete Lifecycle with Multi-Decimal Oracles
      ✔ Should handle complete lifecycle with no depeg
      ✔ Should handle complete lifecycle with depeg
    Event Emissions with Correct Decimal Scaling
      ✔ Should emit CurrentPriceUpdated with base asset decimals
      ✔ Should emit PriceDataUpdated with base asset decimals
      ✔ Should emit SplitToken and RedeemTokens with correct amounts
    Fee Calculations with Low Decimal Precision
      ✔ Should calculate redemption fee correctly with 6 decimals
      ✔ Should handle success fee with 6 decimal precision
      ✔ Should handle small amounts without underflow
    Cross-Oracle Consistency
      ✔ Should produce consistent median across different decimal sources
      ✔ Should handle one oracle deviating significantly
      ✔ Should require minimum sources despite decimal differences
    Depeg Size Calculation with 6 Decimals
      ✔ Should calculate 1% depeg correctly
      ✔ Should calculate 5% depeg correctly
      ✔ Should calculate 10% depeg correctly
    Edge Cases with Low Decimals
      ✔ Should handle maximum allowed price with 6 decimals
      ✔ Should reject price above maximum
      ✔ Should handle minimum price with 6 decimals
```

```
TapirOracle - Cross-Chain Messaging (L1->L2)
  Basic Cross-Chain Setup
    ✔ Should have correct cross-chain configuration
    ✔ Should have oracle set to L1 oracle address on DepegPool
    ✔ Should have L2 messenger configured on DepegPool
    ✔ Should have checkpoints for price writing
  writeCurrentPrice - Cross-Chain Message Flow
    ✔ Should send cross-chain message when writeCurrentPrice is called
    ✔ Should successfully relay message and update price on L2
    ✔ Should emit CurrentPriceUpdated event on L2 after relay
    ✔ Should allow multiple price updates via cross-chain messages
    ✔ Should revert if messenger not configured
  writePriceData - Cross-Chain Message Flow
    ✔ Should send cross-chain message when writePriceData is called
    ✔ Should successfully relay message and update price data on L2
    ✔ Should emit PriceDataUpdated event on L2 after relay
    ✔ Should revert if pool is not in COOLDOWN state
  Security and Edge Cases
    ✔ Should only accept messages from correct xDomainMessageSender
    ✔ Should handle message with zero gas limit
    ✔ Should handle ETH value passed with message
  Comparison: Same-Chain vs Cross-Chain Mode
    ✔ Should update price immediately in same-chain mode
    ✔ Should require manual relay in cross-chain mode*

TapirOracle
  recordApi3Price
    Success Cases
      ✔ Should record API3 price correctly
      ✔ Should handle different price values
      ✔ Should update latestApi3Price on multiple calls
      ✔ Should only be callable by OPERATOR_ROLE
      ✔ Should revert when called by non-operator
      ✔ Should handle int224 to uint192 conversion correctly
      ✔ Should correctly store timestamp as uint64
    Failure Cases
      ✔ Should revert if source is not active
      ✔ Should revert if source is not configured
      ✔ Should revert if both conditions fail
    Edge Cases
      ✔ Should reject zero price value
      ✔ Should handle very small price values
      ✔ Should reject timestamp at 0
      ✔ Should reject future timestamps
  recordChainlinkPrice
    Success Cases
      ✔ Should record Chainlink price correctly
      ✔ Should normalize decimals correctly
      ✔ Should update latestChainlinkPrice on multiple calls
      ✔ Should only be callable by OPERATOR_ROLE
      ✔ Should revert when called by non-operator
    Failure Cases
      ✔ Should revert if source is not active
      ✔ Should revert if source is not configured
      ✔ Should reject zero or negative answer
      ✔ Should reject zero updatedAt
      ✔ Should reject future timestamps
  recordRedStoneClassicPrice
    Success Cases
      ✔ Should record RedStone Classic price correctly
      ✔ Should handle different price values
      ✔ Should normalize decimals correctly
      ✔ Should update latestRedStoneClassicPrice on multiple calls
      ✔ Should only be callable by OPERATOR_ROLE
      ✔ Should revert when called by non-operator
      ✔ Should handle different decimal conversions
    Error Cases
      ✔ Should revert if source is not active
      ✔ Should revert if source is not configured
      ✔ Should reject zero or negative answer
      ✔ Should reject zero updatedAt
      ✔ Should reject future timestamps
```

```
All Four Sources Recording
  ✔ Should correctly record different prices from all four oracle sources
  ✔ Should maintain independence between oracle sources
  ✔ Should correctly normalize different decimal formats to asset decimals
  ✔ Should store correct timestamps for each source
  ✔ Should emit PriceRecorded event with correct data for all sources
View Functions
  ✔ Should return correct sources configuration
  ✔ Should return correct asset metadata
Checkpoint Functionality
  Single Source Checkpoint
    ✔ Should successfully checkpoint with minValidSources=1 and single active source
    ✔ Should handle median calculation correctly with single source
    ✔ Should revert if minValidSources=2 but only one source is available
    ✔ Should revert if minValidSources is configured incorrectly (0)
🌐 Real World Integration Test
  - Should read real API3 price data from live network
  - Should successfully call recordApi3Price multiple times
🌐 Real World Integration Test - Chainlink
  - Should read real Chainlink price data from live network
  - Should successfully call recordChainlinkPrice multiple times
🌐 Real World Integration Test - RedStone Classic
  - Should read real RedStone Classic price data from live network
  - Should successfully call recordRedStoneClassicPrice multiple times
    Auxiliary tests
      Downcasting & uint192 Price Bounds
        ✔ Should reject API3 price that exceeds uint192 max after normalization
        ✔ Should reject Chainlink price that exceeds uint192 max after normalization
        ✔ Should reject RedStone Classic price that exceeds uint192 max after normalization
        ✔ Should reject values that overflow after decimal conversion scaling up
    Ring Buffer and Lookback Period
      ✔ Should handle 364 daily checkpoints + 24 hourly checkpoints with 24h lookback period
      ✔ Should correctly count checkpoints inside lookback period AND outside when not enough checkpoints
are available inside
    Daily Aggregation Logic
      ✔ Should correctly aggregate prices by day with varying checkpoint counts (1, 2, 3, 4, 5 prices per
day)
      ✔ Should handle single checkpoint per day correctly for 5 days
      ✔ Should use smaller value when exactly 2 prices recorded on a day
      ✔ Should correctly compute median for 3+ prices on same day

  TapirPtrwOracle
    recordApi3Price
      Success Cases
        ✔ Should record API3 price correctly
        ✔ Should handle different price values
        ✔ Should update latestApi3Price on multiple calls
        ✔ Should only be callable by OPERATOR_ROLE
        ✔ Should revert when called by non-operator
        ✔ Should handle int224 to uint192 conversion correctly
        ✔ Should correctly store timestamp as uint64
      Failure Cases
        ✔ Should revert if source is not active
        ✔ Should revert if source is not configured
        ✔ Should revert if both conditions fail
      Edge Cases
        ✔ Should reject zero price value
        ✔ Should handle very small price values
        ✔ Should reject timestamp at 0
        ✔ Should reject future timestamps
    recordChainlinkPrice
      Success Cases
        ✔ Should record Chainlink price correctly
        ✔ Should normalize decimals correctly
        ✔ Should update latestChainlinkPrice on multiple calls
        ✔ Should only be callable by OPERATOR_ROLE
        ✔ Should revert when called by non-operator
      Failure Cases
        ✔ Should revert if source is not active
        ✔ Should revert if source is not configured
        ✔ Should reject zero or negative answer
        ✔ Should reject zero updatedAt
```

```
    ✔ Should reject future timestamps
recordRedStoneClassicPrice
  Success Cases
    ✔ Should record RedStone Classic price correctly
    ✔ Should handle different price values
    ✔ Should normalize decimals correctly
    ✔ Should update latestRedStoneClassicPrice on multiple calls
    ✔ Should only be callable by OPERATOR_ROLE
    ✔ Should revert when called by non-operator
    ✔ Should handle different decimal conversions
  Error Cases
    ✔ Should revert if source is not active
    ✔ Should revert if source is not configured
    ✔ Should reject zero or negative answer
    ✔ Should reject zero updatedAt
    ✔ Should reject future timestamps
All Four Sources Recording
  ✔ Should correctly record different prices from all four oracle sources
  ✔ Should maintain independence between oracle sources
  ✔ Should correctly normalize different decimal formats to asset decimals
  ✔ Should store correct timestamps for each source
  ✔ Should emit PriceRecorded event with correct data for all sources
View Functions
  ✔ Should return correct sources configuration
  ✔ Should return correct asset metadata
Checkpoint Functionality
  Single Source Checkpoint
    ✔ Should successfully checkpoint with minValidSources=1 and single active source
    ✔ Should handle median calculation correctly with single source
    ✔ Should revert if minValidSources=2 but only one source is available
    ✔ Should revert if minValidSources is configured incorrectly (0)
🌐 Real World Integration Test
  - Should read real API3 price data from live network
  - Should successfully call recordApi3Price multiple times
🌐 Real World Integration Test - Chainlink
  - Should read real Chainlink price data from live network
  - Should successfully call recordChainlinkPrice multiple times
🌐 Real World Integration Test - RedStone Classic
  - Should read real RedStone Classic price data from live network
  - Should successfully call recordRedStoneClassicPrice multiple times
Auxiliary tests
  Downcasting & uint192 Price Bounds
    ✔ Should reject API3 price that exceeds uint192 max after normalization
    ✔ Should reject Chainlink price that exceeds uint192 max after normalization
    ✔ Should reject RedStone Classic price that exceeds uint192 max after normalization
    ✔ Should reject values that overflow after decimal conversion scaling up
Ring Buffer and Lookback Period
  - Should handle 364 daily checkpoints + 24 hourly checkpoints with 24h lookback period
    - Should correctly count checkpoints inside lookback period AND outside when not enough checkpoints
are available inside
    Daily Aggregation Logic
    - Should correctly aggregate prices by day with varying checkpoint counts (1, 2, 3, 4, 5 prices per
day)
    - Should handle single checkpoint per day correctly for 5 days
    - Should use smaller value when exactly 2 prices recorded on a day
    - Should correctly compute median for 3+ prices on same day

  TapirPtrwOracle - Mainnet Fork Integration Test

================================================================================
MAINNET FORK TEST: TapirPtrwOracle with Expired Pendle PT
================================================================================
RPC: https://eth-mainnet.g.alchemy.com/v2/AAXvBdsmb45SC...
PT Token: 0xd4e75971eaf78a8d93d96df530f1fff5f9f53288
YT Token: 0x1a65eB80a2ac3ea6E41D456DdD6E9cC5728BEf7C
pufETH (Base): 0xd9a442856c234a39a81a089c06451ebaa4306a72
Pendle Router: 0x888888888889758F76e7103c6CbF23ABbF58F946
PT Expiry: 2024-09-26T00:00:00.000Z
================================================================================

Connected to network with Chain ID: 31337
Current block: 23997443
```

```
Block timestamp: 2025-12-12T15:30:47.000Z
PT is expired: true
PT Total Supply: 51.868453045866268776 PT
    Oracle Deployment and Configuration
Deployed TapirPtrwOracle at: 0xedEe4820327176Bd433d13421DD558A7191193Aa
Pendle config verified
      ✔ Should deploy TapirPtrwOracle with Pendle config
    PTRW Resolution Tests
      ✔ Should fail resolvePtrw when called by non-operator

Oracle PT Balance: 0.0 PT
      ✔ Should fail resolvePtrw with no PT balance

PT Holder Balance: 0.153498364737 PT
Transferring 0.0767491823685 PT to oracle...
Oracle PT Balance: 0.0767491823685 PT

Calling resolvePtrw() as operator...
PTRW Resolved! Gas used: 322724

PTRW Results:
   ptrwErv (Expected): 0.0767491823685 PT
   ptrwArv (Actual): 0.072168460781832765 pufETH
   Ratio: 0.9403156952907515
   ptrwResolved: true
      ✔ Should resolve PTRW with a PT balance

First resolvePtrw() call...
First resolution - ERV: 0.019187295592125, ARV: 0.018042115195458191
      ✔ Should allow resolvePtrw to be called multiple times with new PT deposits
    Manual Backup Resolution Test

PT isExpired: true
      ✔ Should allow admin to manually resolve PTRW for expired PT
      ✔ Should reject manual resolution when PT is not expired
      ✔ Should reject manual resolution from non-admin
      ✔ Should reject manual resolution with zero ERV
      ✔ Should reject manual resolution when PT balance exists
      ✔ Should reject manual resolution if already resolved
    PTRW Depeg Factor Calculation
      ✔ Should correctly calculate depeg factor when ARV < ERV

  TimeDecay
    calculateDecay
      Basic functionality
        ✔ Should return PRECISION (1e18) at t=0 (start of period)
        ✔ Should return MIN_DECAY in the end (t=T)
        ✔ Should decrease monotonically over time
      ✔ Should calculate approximately sqrt(0.5) at t=T/2
      ✔ Should calculate approximately sqrt(0.25) at t=3T/4
      ✔ Should calculate approximately sqrt(0.75) at t=T/4
    MIN_DECAY threshold
      ✔ Should apply MIN_DECAY threshold when decay would be below minimum
      ✔ Should find the approximate time when MIN_DECAY threshold is reached
    Different durations
      ✔ Should work with 1 day duration
      ✔ Should work with 30 days duration
      ✔ Should work with 90 days duration
      ✔ Should work with 365 days duration
      ✔ Should work with very short duration
    Precision and scaling
      ✔ Should maintain 1e18 precision in result
      ✔ Should not overflow with maximum practical duration
      ✔ Should handle very small remaining ratios correctly
    Edge cases
      ✔ Should handle elapsedTime = 0 correctly
      ✔ Should handle elapsedTime = totalDuration correctly
      ✔ Should handle very small durations (1 second)
      ✔ Should revert if totalDuration is 0
      ✔ Should revert if elapsedTime > totalDuration
    Mathematical properties
      ✔ Should satisfy: d(0) = 1
```

```
      ✔ Should satisfy: d(t1) > d(t2) for t1 < t2
    calculateDecayDiff
      Basic functionality
        ✔ Should return positive difference when t1 < t2
        ✔ Should return negative difference when t1 > t2
        ✔ Should return zero when t1 = t2
        ✔ Should calculate correct difference between start and halfway
      Edge cases
        ✔ Should handle both times at start (t=0)
        ✔ Should handle both times at end (t=T)
        ✔ Should handle large time difference
        ✔ Should handle very small time differences

·-------------------------------------------------------------|---------------------------|-----------|-----
-------------------------·
|                  [90mSolc version: 0.8.27[39m                  ·  [90mOptimizer enabled: true[39m
·  [90mRuns: 1[39m  ·  [90mBlock limit: 30000000 gas[39m  |
·····························------------------------------------------------------|···························|···········|
·····························
|  [32m[1mMethods[22m[39m
|
·····························|·····························|·············|·············|···········|
·············|·············
|  [1mContract[22m                 ·  [1mMethod[22m                      ·  [32mMin[39m        ·
[32mMax[39m        ·  [32mAvg[39m      ·  [1m# calls[22m      ·  [1musd (avg)[22m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mApi3ReaderProxyMock[39m       ·  setData                    ·      [36m26706[39m  ·
[31m26958[39m  ·    26779  ·          [90m540[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mApi3ReaderProxyMock[39m       ·  setTimestamp               ·          –  ·          –  ·
23678  ·            [90m2[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mApi3ReaderProxyMock[39m       ·  setValue                   ·          –  ·          –  ·
26509  ·            [90m2[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mChainlinkAggregatorMock[39m   ·  setAnswer                  ·      [36m23648[39m  ·
[31m43488[39m  ·    24015  ·          [90m392[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mChainlinkAggregatorMock[39m   ·  setData                    ·      [36m23971[39m  ·
[31m31667[39m  ·    30579  ·          [90m120[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mChainlinkAggregatorMock[39m   ·  setUpdatedAt               ·      [36m23578[39m  ·
[31m26378[39m  ·    26349  ·          [90m392[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegFactory[39m              ·  deployDepeg                ·    [36m3645276[39m  ·
[31m3687998[39m  ·  3662422  ·          [90m258[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegFactory[39m              ·  renounceOwnership          ·          –  ·          –  ·
23165  ·            [90m1[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegFactory[39m              ·  transferOwnership          ·          –  ·          –  ·
28515  ·            [90m2[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegPool[39m                 ·  executeOracleChange        ·          –  ·          –  ·
30366  ·            [90m5[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegPool[39m                 ·  grantRole                  ·      [36m51033[39m  ·
[31m51429[39m  ·    51426  ·          [90m157[39m  ·          [32m[90m–[32m[39m  |
·····························|·····························|·············|·············|···········|
·············|·············
|  [90mDepegPool[39m                 ·  pause                      ·      [36m35500[39m  ·
```

```
[31m37793 [39m  ·   35755  ·          [90m9 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  proposeOracleChange         ·       [36m76631 [39m  ·
[31m76643 [39m  ·   76640  ·          [90m7 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  redeemTokens                ·       [36m74358 [39m  ·
[31m88708 [39m  ·   77671  ·         [90m26 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  rescueErc20                 ·       [36m55809 [39m  ·
[31m67110 [39m  ·   63343  ·          [90m3 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  resolvePriceDepeg           ·       [36m43057 [39m  ·
[31m43883 [39m  ·   43490  ·         [90m40 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  revokeRole                  ·       [36m28044 [39m  ·
[31m30428 [39m  ·   28839  ·          [90m3 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  setAuthorisedRouter         ·       [36m27021 [39m  ·
[31m48945 [39m  ·   44553  ·          [90m5 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  setCooldownDuration         ·            –  ·            –  ·
30746  ·            [90m3 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  setMinPriceAge              ·            –  ·            –  ·
29996  ·            [90m3 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  setRedemptionFeeBp          ·       [36m30715 [39m  ·
[31m33020 [39m  ·   31183  ·          [90m5 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  setSuccessFeeBp             ·       [36m29879 [39m  ·
[31m32196 [39m  ·   30357  ·          [90m5 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  splitToken                  ·       [36m81991 [39m  ·
[31m169691 [39m  ·  145236  ·        [90m100 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  sweepFeesToTreasury         ·            –  ·            –  ·
65611  ·            [90m3 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  unpause                     ·            –  ·            –  ·
29913  ·            [90m4 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  unSplitTokens               ·       [36m70611 [39m  ·
[31m85852 [39m  ·   77035  ·         [90m19 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  updateCurrentPrice          ·       [36m34737 [39m  ·
[31m37603 [39m  ·   36162  ·         [90m14 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegPool [39m              ·  updatePriceData             ·       [36m98975 [39m  ·
[31m101877 [39m  ·   99265  ·         [90m47 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegToken [39m             ·  approve                     ·       [36m26374 [39m  ·
[31m46274 [39m  ·   45579  ·         [90m86 [39m  ·          [32m [90m– [32m [39m  │
······························|································|··············|··············|···········|
···············|··············
|  [90mDepegToken [39m             ·  burn                        ·            –  ·            –  ·
```

```
34143  ·             [90m4 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mDepegToken [39m                  ·  mint                         ·      [36m51083 [39m  ·
 [31m68195 [39m  ·     66294  ·            [90m9 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mDepegToken [39m                  ·  transfer                     ·      [36m29464 [39m  ·
 [31m53960 [39m  ·     48094  ·            [90m5 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mMockCrossDomainMessenger [39m  ·  setXDomainMessageSender      ·      [36m24031 [39m  ·
 [31m43931 [39m  ·     41085  ·           [90m14 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mSimpleMockOracle [39m            ·  setDepegPool                 ·           -  ·           -  ·
43793  ·             [90m6 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  checkpoint                   ·      [36m69890 [39m  ·
 [31m90751 [39m  ·     73564  ·          [90m501 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  grantRole                    ·      [36m51329 [39m  ·
 [31m51341 [39m  ·     51331  ·          [90m138 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  recordApi3Price              ·      [36m38275 [39m  ·
 [31m58406 [39m  ·     42890  ·          [90m524 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  recordChainlinkPrice         ·      [36m42891 [39m  ·
 [31m62791 [39m  ·     47076  ·          [90m465 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  recordRedStoneClassicPrice   ·      [36m43150 [39m  ·
 [31m63050 [39m  ·     59043  ·           [90m27 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  recordTellorPrice            ·           -  ·           -  ·
62835  ·             [90m7 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  setConfig                    ·      [36m37767 [39m  ·
 [31m57907 [39m  ·     48846  ·           [90m57 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  setSources                   ·      [36m43667 [39m  ·
 [31m47199 [39m  ·     44203  ·            [90m8 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  writeCurrentPrice            ·      [36m48019 [39m  ·
 [31m226498 [39m  ·    192242  ·           [90m13 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirOracle [39m                 ·  writePriceData               ·     [36m143879 [39m  ·
 [31m2364636 [39m  ·    421979  ·           [90m11 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirPtrwOracle [39m             ·  checkpoint                   ·      [36m70115 [39m  ·
 [31m87215 [39m  ·     78665  ·            [90m4 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirPtrwOracle [39m             ·  grantRole                    ·           -  ·           -  ·
51373  ·            [90m91 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirPtrwOracle [39m             ·  manualBackupResolvePtrw      ·           -  ·           -  ·
100456  ·             [90m3 [39m  ·           [32m [90m- [32m [39m  │
·····························|······························|·············|·············|···········|
···············|··············
|  [90mTapirPtrwOracle [39m             ·  recordApi3Price              ·      [36m41374 [39m  ·
```

```
[31m58474 [39m  ·  54199  ·  [90m24 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · recordChainlinkPrice  · [36m45759 [39m  ·
[31m62859 [39m  ·  59195  ·  [90m14 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · recordRedStoneClassicPrice  · [36m46238 [39m  ·
[31m63338 [39m  ·  59314  ·  [90m17 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · recordTellorPrice  ·  -  ·  -  ·
62925  ·  [90m7 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · resolvePtrw  · [36m257312 [39m  ·
[31m322724 [39m  ·  286626  ·  [90m7 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · setConfig  · [36m37811 [39m  ·
[31m55187 [39m  ·  42170  ·  [90m4 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mTapirPtrwOracle [39m  · setSources  · [36m43711 [39m  ·
[31m43951 [39m  ·  43817  ·  [90m7 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mUSDC6Mock [39m  · approve  · [36m46262 [39m  ·
[31m46274 [39m  ·  46273  ·  [90m66 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mUSDC6Mock [39m  · mint  · [36m51020 [39m  ·
[31m68120 [39m  ·  61397  ·  [90m84 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mWtETHMock [39m  · approve  · [36m46262 [39m  ·
[31m46274 [39m  ·  46273  ·  [90m330 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [90mWtETHMock [39m  · mint  · [36m33968 [39m  ·
[31m68156 [39m  ·  59496  ·  [90m294 [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| [32m [1mDeployments [22m [39m  ·
·  [1m% of limit [22m  ·  |
·····|·····|·····|·····|·····|·····|·····
| Api3ReaderProxyMock  · [36m161784 [39m  · [31m161856 [39m
·  161849  ·  [90m0.5 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| ChainlinkAggregatorMock  · [36m166671 [39m  · [31m186811 [39m
·  186116  ·  [90m0.6 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| DepegFactory  ·  -  ·  -  ·  4241217  ·
[90m14.1 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| DepegToken  · [36m579824 [39m  · [31m579908 [39m
·  579866  ·  [90m1.9 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| MockCrossDomainMessenger  ·  -  ·  -  ·  679196  ·
[90m2.3 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| MockDepegPoolForOracle  ·  -  ·  -  ·  119235  ·
[90m0.4 % [39m  ·  [32m [90m- [32m [39m  |
·····|·····|·····|·····|·····|·····|·····
| SimpleMockOracle  ·  -  ·  -  ·  250459  ·
```

```
[90m0.8 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  TapirOracle                                            ·    [36m2650452[39m  ·    [31m2690504[39m
·  2672480  ·        [90m8.9 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  TapirPtrwOracle                                        ·    [36m3205417[39m  ·    [31m3206221[39m
·  3205501  ·       [90m10.7 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  TimeDecayTest                                          ·          -  ·          -  ·   223228  ·
[90m0.7 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  Token                                                  ·          -  ·          -  ·   508218  ·
[90m1.7 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  USDC6Mock                                              ·          -  ·          -  ·   493658  ·
[90m1.6 %[39m  ·          [32m[90m-[32m[39m  │
·························································|·············|·············|···········|
··············|··············
|  WtETHMock                                              ·          -  ·          -  ·   493139  ·
[90m1.6 %[39m  ·          [32m[90m-[32m[39m  │
·---------------------------------------------------------|-------------|-------------|-----------|----
-----------|--------------·

  371 passing (34s)
  18 pending
```

Code Coverage

To get the code coverage of the projects test suite run the following commands:

```
npx hardhat coverage
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	82.64	71.57	86.3	85.04	
DepegFactory.sol	100	53.33	100	100	
DepegPool.sol	98.2	81.88	100	98.13	314,440,461
DepegToken.sol	100	100	100	100	
TapirOracle.sol	97.35	79.85	93.1	97.34	... 297,480,571
TapirPtrwOracle.sol	71.43	61.11	75	72.09	... 174,177,180
TapirRouter.sol	0	0	0	0	... 197,212,213
contracts/interfaces/	100	100	100	100	
IDepegFactory.sol	100	100	100	100	
IDepegPool.sol	100	100	100	100	
IDepegToken.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
IPMarketFactoryV3.sol	100	100	100	100	
IPendleRouter.sol	100	100	100	100	
ISimpleAMM.sol	100	100	100	100	
IStableSwap.sol	100	100	100	100	
ITapirOracle.sol	100	100	100	100	
ITapirPtrwOracle.sol	100	100	100	100	
IUniswapV3SwapCallback.sol	100	100	100	100	
IV3SwapRouter.sol	100	100	100	100	
ItBase.sol	100	100	100	100	
contracts/libraries/	23.26	10.34	50	16.67	
TimeDecay.sol	100	100	100	100	
UniswapV3Math.sol	0	0	0	0	... 143,151,152
contracts/mock/	27.12	5	45	34.62	
AMMTwapMock.sol	0	100	0	0	14,15,19,23
Api3ReaderProxyMock.sol	100	100	100	100	
ChainlinkAggregatorMock.sol	50	100	83.33	88.89	27
ChainlinkMock.sol	0	100	0	0	... 32,36,40,44
CurvePoolMock.sol	0	0	0	0	... 31,35,36,37
ERC20.sol	100	100	100	100	
LPMock.sol	0	100	0	0	... 24,28,32,36
LiquidityPoolMock.sol	0	100	0	0	... 19,23,27,31
MockCrossDomainMessenger.sol	26.92	6.25	71.43	24.24	... 134,138,142
MockDepegPoolForOracle.sol	100	100	50	100	
SimpleMockOracle.sol	50	100	50	50	29,33,34,38
TimeDecayTest.sol	100	100	100	100	
USDC6Mock.sol	100	100	100	100	
assetMock.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
All files	70.11	61.23	67.15	70.25	

Changelog

- 2025-12-15 - Initial report
- 2026-01-06 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.